Основы анализа данных

Качество данных на примере HMDA

Проверка и очистка ипотечной выгрузки

Знакомая таблица HMDA

decisions.csv — записи о покупке жилья
Вашингтон, 2023 год

source_row	action_taken	loan_amount	income
2	1	275000	72
3	1	585000	174
5	1	275000	250
7	1	295000	63

Ноутбук: B02

Словарь полей

Семь столбцов исходного файла

Поле	Значение
source_row	Номер строки исходного снимка
activity_year	Отчётный год
lei	Код организации
county_code	Код территории
action_taken	Исход: 1 — выдача, 2 — одобрение, 3 — отказ
loan_amount	Сумма кредита, USD
income	Доход за год, тыс. USD

Ноутбук: B03

Правила для нашего расчёта

Одобрения: исходы 1 и 2 среди исходов 1–3

Поле	Проверка	Действие
source_row	Известен, без повторной доставки	Разобрать повторы
action_taken	Код 1, 2 или 3	Отложить нарушение
loan_amount	Число больше нуля	Отложить нарушение
county_code	Код 11001	Уточнить территорию
income	Может отсутствовать	Сохранить и посчитать пропуски

Ноутбук: B04

Профиль пропусков

isna() возвращает True там, где значение неизвестно

sum() считает эти отметки по столбцам

```
unknown_income = decisions["income"].isna()
decisions.loc[unknown_income, ["action_taken", "income"]].head()
```

Ноутбук: B05

Пропуск и ноль

Известные доходы: 100 и 200; третий неизвестен

Среднее известных — 150, после fillna(0) — 100

```
illustration = pd.Series([100.0, 200.0, float("nan")])
illustration.mean()
illustration.fillna(0).mean()
```

Ноутбук: B06

Доля пропусков по исходам

Для каждой группы: пропуски / все её записи

action_taken	Все записи	Нет income	Доля
1	5956	244	4,10%
2	183	33	18,03%
3	691	65	9,41%

Ноутбук: B07

Удаление строк с неизвестным доходом

dropna(subset=["income"]) оставляет известный income

Меняется и знаменатель, и состав записей

```
complete_income = decisions.dropna(subset=["income"])
print(len(decisions), len(complete_income))
```

Ноутбук: B08

Причина отсутствия значения

В файле виден пропуск, но не его причина

Следующий шаг — описание источника и вопрос поставщику

Учебная копия выгрузки

training_export.csv — копия decisions.csv с учебными дефектами
Читаем как string, чтобы видеть исходный текст

source_row	Поле	Было в оригинале	В учебной копии
2	loan_amount	275000.0	oops
3	action_taken	1	88
5	income	250.0	BAD
7	county_code	11001	" 11001 "

Ноутбук: B10

Текст в число и ошибки преобразования

to_numeric(..., errors="coerce") возвращает числа

Нераспознанный текст превращается в пропуск

```
numeric_amount = pd.to_numeric(export["loan_amount"], errors="coerce")
parse_error = export["loan_amount"].notna() & numeric_amount.isna()
```

Ноутбук: B11

Пробелы и коды категорий

str.strip() убирает внешние пробелы

isin([1, 2, 3]) проверяет допустимые исходы

```
normalized_county = export["county_code"].str.strip()
invalid_outcome = ~outcome_num.isin([1, 2, 3])
```

Ноутбук: B12

Единицы измерения

Числовой тип не задаёт единицу

Поле	Единица	Пример
loan_amount	USD	275000
income	тыс. USD в год	72
income_usd	USD в год	72000

Ноутбук: B13

Повторы организации и строки

duplicated() отмечает повторные вхождения

Что сравниваем	Повторяется	Наше действие
lei	Организация в разных записях	Сохраняем строки
Шесть полей без source_row	Опубликованные значения	Показываем совпадения
source_row в экспорте	Доставка исходной строки	Сравниваем всю запись

Ноутбук: B14

Подтверждённые повторы доставки

Один source_row: сначала сравнить все значения

Разные значения одного ключа — остановка

```
repeated = export.loc[known_key].drop_duplicates()
assert repeated["source_row"].is_unique
```

Ноутбук: B15

Необычно большая сумма

Q1 и Q3 — границы средней половины значений

IQR = Q3 – Q1; порог = Q3 + 1,5 × IQR

```
q1, q3 = amount.quantile([0.25, 0.75])
iqr = q3 - q1
upper_flag = q3 + 1.5 * iqr
```

Ноутбук: B16

Среднее и медиана суммы

Среднее использует каждую сумму

Медиана делит упорядоченные значения пополам

```
print(amount.mean(), amount.median())
without_tail = decisions.loc[amount.le(upper_flag)]
```

Ноутбук: B17

Пересечение условий

Нет дохода: 342; отказ: 691

Совпали оба условия: 65

```
(missing_mask | denial_mask).sum()
(missing_mask & denial_mask).sum()
```

Ноутбук: B18

Карантин и баланс

Карантин — отдельная таблица строк с нарушениями
Причина сохраняется вместе с исходными значениями

Часть результата	Строк	Что произошло
Входной экспорт	6832	До исправлений
Повторные доставки	2	Оставлено первое вхождение
Карантин	2	Сумма oops или исход 88
Подготовленные записи	6828	Удовлетворяют правилам

Функция очистки

clean_export(source) → clean, quarantine, audit

На входе текстовая таблица, на выходе новые объекты

```
clean, quarantine, audit = clean_export(export)
print(audit)
quarantine[["source_row", "loan_amount", "action_taken", "reason"]]
```

Ноутбук: B20

Проверки результата и входа

assert останавливает выполнение при ложном условии

assert_frame_equal сравнивает две таблицы

```
assert_frame_equal(export, raw_copy)
assert len(export) == len(clean) + len(quarantine) + audit["replays"]
```

Ноутбук: B21

Показатель после очистки

Счётчик и знаменатель показываем для каждого слоя

Известный доход — отдельный отбор

```
show_approval("Оригинал", decisions)
show_approval("После учебной очистки", clean)
```

Ноутбук: B22

Файлы результата и происхождение

Исходный CSV → код → clean, карантин и журнал
Отчёт связывает файлы с вопросом и правилами

Файл	Содержание
clean.csv	Подготовленные записи
quarantine.csv	Исходные спорные строки и reason
quality_report.json	Количество строк и ошибок
README.md	Вопрос, правила, результаты, ограничения

Практика очистки

Сначала осмотр и правила, затем преобразования

В конце — проверка, файлы и короткий отчёт